UNIVERSIDADE SANTO AMARO

CURSO SUPERIOR DE TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

BIANCA GOMES DANTAS DE ALMEIDA — RA 5665248

GIOVANNA GOMES SILVA — RA 5565791

JAILTON SANTOS DE MATOS — RA 5773083

LUCAS ALVES DE SOUZA — RA 5740304

LUCAS NEVES DOS SANTOS — RA 5718422

GERENCIADOR DE EVENTOS ACADÊMICOS:

aplicação web com inteligência artificial para inscrições, presença e certificados

SÃO PAULO

2026

BIANCA GOMES DANTAS DE ALMEIDA — RA 5665248
GIOVANNA GOMES SILVA — RA 5565791
JAILTON SANTOS DE MATOS — RA 5773083
LUCAS ALVES DE SOUZA — RA 5740304
LUCAS NEVES DOS SANTOS — RA 5718422

GERENCIADOR DE EVENTOS ACADÊMICOS:

aplicação web com inteligência artificial para inscrições, presença e certificados

Projeto Integrador apresentado ao Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas, turma 4A, da Universidade Santo Amaro (UNISA), como requisito parcial para aprovação.

Orientador: Prof. Rafael Guem Murakami

SÃO PAULO
2026

RESUMO

A organização de eventos acadêmicos costuma depender de planilhas, formulários avulsos e certificados produzidos manualmente, o que consome tempo e compromete a confiabilidade dos registros de participação. Este trabalho apresenta o desenvolvimento do Gerenciador de Eventos Acadêmicos, aplicação web que reúne em uma única plataforma a publicação de eventos, a inscrição com controle de vagas e fila de espera, o registro de presença, a emissão de certificados com código de autenticidade e a coleta de avaliações. Parte-se da hipótese de que a centralização dessas etapas, com regras de negócio verificadas automaticamente e com a inteligência artificial como apoio supervisionado ao cadastro, reduz o trabalho manual sem retirar do organizador o controle sobre o evento. Trata-se de uma pesquisa aplicada, conduzida por especificação de requisitos, modelagem, implementação incremental e verificação por testes automatizados de unidade, de integração e de ponta a ponta. A solução foi construída com Java e Spring Boot no servidor, React no navegador e banco de dados PostgreSQL, e publicada em serviços de nuvem gratuitos. Como resultado, obteve-se um sistema funcional que executa o fluxo completo, da criação do evento à validação pública do certificado, no qual o certificado é liberado apenas a quem teve presença registrada e após o término do evento, e no qual a falha do serviço de inteligência artificial não interrompe o cadastro. Conclui-se que a proposta atende aos objetivos traçados, embora a redução de tempo ainda não tenha sido medida com usuários reais.

Palavras-chave: Eventos acadêmicos. Inteligência artificial. Certificado digital. Desenvolvimento web.

ABSTRACT

Organizing academic events often relies on spreadsheets, scattered forms and manually produced certificates, which takes time and undermines the reliability of attendance records. This work presents the development of the Academic Event Manager, a web application that brings together, in a single platform, event publishing, registration with seat control and a waiting list, attendance recording, the issuing of certificates with an authenticity code, and the collection of feedback. The hypothesis is that centralizing these steps, with automatically verified business rules and with artificial intelligence as supervised support for event registration, reduces manual work without taking control of the event away from the organizer. This is applied research, carried out through requirements specification, modeling, incremental implementation and verification by automated unit, integration and end-to-end tests. The solution was built with Java and Spring Boot on the server, React in the browser and a PostgreSQL database, and was deployed on free cloud services. As a result, a working system was obtained that runs the complete flow, from event creation to the public validation of the certificate, in which the certificate is released only to those whose attendance was recorded and only after the event ends, and in which a failure of the artificial intelligence service does not interrupt event registration. It is concluded that the proposal meets its objectives, although the time savings have not yet been measured with real users.

Keywords: Academic events. Artificial intelligence. Digital certificate. Web development.

SUMÁRIO

1 INTRODUÇÃO..... 5
1.1 Problema..... 5
1.2 Hipótese..... 6
1.3 Objetivos..... 6
1.3.1 Objetivo geral..... 6
1.3.2 Objetivos específicos..... 6
1.4 Justificativa..... 7
2 REFERENCIAL TEÓRICO..... 8
2.1 Arquitetura em camadas e código limpo..... 8
2.2 Modelagem com a linguagem unificada..... 8
2.3 Autenticação por token..... 9
2.4 Modelos de linguagem e saída estruturada..... 9
2.5 Testes automatizados..... 9
3 METODOLOGIA..... 10
3.1 Tipo de pesquisa..... 10
3.2 Etapas e cronograma..... 10
3.3 Tecnologias..... 11
3.4 Especificação e rastreabilidade..... 11
3.5 Verificação e publicação..... 12
4 RESULTADOS E DISCUSSÃO..... 13
4.1 Requisitos e regras de negócio..... 13
4.2 Arquitetura do sistema..... 15
4.3 Modelo de dados..... 15
4.4 Inscrições e fila de espera..... 16
4.5 Presença e certificação..... 16
4.6 Inteligência artificial..... 18
4.7 Interface..... 18
4.8 Testes e publicação..... 19
4.9 Discussão..... 20
5 CONCLUSÃO..... 21
REFERÊNCIAS..... 22

1 INTRODUÇÃO

Palestras, workshops, semanas temáticas e congressos internos fazem parte da rotina das instituições de ensino superior e envolvem etapas que vão da divulgação à certificação dos participantes. Em muitos casos, essas etapas ainda são conduzidas de forma dispersa: as inscrições chegam por formulários avulsos, a presença é anotada em listas de papel ou em planilhas e os certificados são produzidos um a um, depois do evento.

Diante desse cenário, este trabalho propõe o Gerenciador de Eventos Acadêmicos, uma aplicação web que reúne em uma única plataforma o cadastro e a publicação de eventos, a inscrição com controle de vagas e fila de espera, o registro de presença, a emissão de certificados com código de autenticidade e a coleta de avaliações. Para reduzir o esforço de cadastro, o sistema emprega inteligência artificial na extração dos dados do evento a partir de um texto livre, mantendo o organizador como responsável pela revisão final. O código-fonte do sistema está disponível no repositório https://github.com/Jayzmatos22/GerenciadorDeEventos.

Além desta introdução, que apresenta o problema, a hipótese, os objetivos e a justificativa, o trabalho está organizado em quatro seções. A seção 2 reúne o referencial teórico que fundamenta as escolhas técnicas; a seção 3 descreve a metodologia e o cronograma; a seção 4 apresenta e discute os resultados, da especificação à publicação do sistema; e a seção 5 traz a conclusão, as limitações e as sugestões de continuidade.

1.1 Problema

O arranjo disperso descrito acima produz três dificuldades concretas. A primeira é o controle de vagas: quando as inscrições ultrapassam a capacidade do local, a fila de espera é mantida à mão, e a desistência de um inscrito raramente chega a quem estava aguardando. A segunda é a confiabilidade da participação: sem um registro de presença associado à inscrição, não há como distinguir quem compareceu de quem apenas se inscreveu. A terceira é a certificação: os certificados são produzidos depois do evento, sem comprovação de presença e sem um meio de terceiros conferirem sua autenticidade.

O problema que orienta este trabalho pode, assim, ser formulado como uma pergunta: como reunir, em uma única ferramenta, as etapas de um evento acadêmico de modo que os registros de participação sejam confiáveis e que o esforço de cadastro e de certificação diminua?

1.2 Hipótese

Adota-se como hipótese que uma aplicação web que centralize essas etapas, com regras de negócio explícitas e verificadas automaticamente, e que empregue a inteligência artificial apenas como apoio supervisionado ao cadastro, reduz o trabalho manual sem retirar do organizador o controle sobre o evento. A hipótese tem duas partes: a primeira, de natureza técnica, é a viabilidade de garantir as regras de vagas, presença e certificação por meio do próprio sistema; a segunda é que essa centralização efetivamente reduz o tempo gasto pelos organizadores.

1.3 Objetivos

1.3.1 Objetivo geral

Desenvolver uma aplicação web para a gestão completa de eventos acadêmicos, capaz de administrar inscrições com controle de vagas e fila de espera, registrar a presença dos participantes e emitir certificados digitais com validação pública, incorporando recursos de inteligência artificial ao processo de cadastro.

1.3.2 Objetivos específicos

a) disponibilizar autenticação e controle de acesso diferenciado para participantes e organizadores;

b) permitir a criação e a divulgação de eventos, tanto por preenchimento manual quanto por extração automática de dados a partir de texto livre;

c) controlar a ocupação das vagas, incluindo a formação de fila de espera e a promoção automática em caso de desistência;

d) registrar a presença dos participantes durante a realização do evento;

e) emitir certificados digitais em formato PDF, vinculados à presença e ao término do evento e dotados de código de autenticidade verificável publicamente;

f) coletar avaliações após o encerramento do evento e sintetizá-las automaticamente em um resumo consolidado;

g) assegurar a continuidade da operação em caso de indisponibilidade do serviço de inteligência artificial.

1.4 Justificativa

A relevância do trabalho está, em primeiro lugar, na rastreabilidade que ele introduz. Quando o certificado depende de um registro de presença feito pelo organizador e pode ser conferido por um código público, a participação deixa de ser uma declaração e passa a ser um fato verificável, o que interessa tanto à instituição quanto ao participante, que passa a dispor de um documento cuja autenticidade qualquer pessoa pode confirmar.

Em segundo lugar, o trabalho ataca a etapa mais repetitiva da organização. A extração automática dos dados de um convite em texto livre reduz o preenchimento de formulários, e a fila de espera com promoção automática dispensa o acompanhamento manual das desistências. Por fim, o projeto tem valor formativo para a equipe, por exigir a aplicação integrada de especificação de requisitos, modelagem, arquitetura de software, banco de dados, segurança, testes automatizados e publicação em nuvem, conteúdos centrais do curso de Análise e Desenvolvimento de Sistemas.

2 REFERENCIAL TEÓRICO

Esta seção apresenta os conceitos que fundamentam as escolhas técnicas do projeto.

2.1 Arquitetura em camadas e código limpo

Fowler (2002) descreve a divisão de uma aplicação em camadas como forma de separar responsabilidades: a apresentação trata da interação com o usuário, o domínio concentra as regras de negócio e a camada de dados cuida da persistência. Cada camada depende apenas da imediatamente inferior, o que permite modificar uma delas sem reescrever as demais e facilita testar as regras de negócio isoladamente. O mesmo autor apresenta a camada de serviço como o ponto que define as operações disponíveis da aplicação e coordena as respostas a cada operação.

Martin (2008) complementa essa visão no nível do código, ao defender nomes que revelem a intenção, funções pequenas e com uma única responsabilidade e a ausência de duplicação. Para o autor, o código é lido muito mais vezes do que é escrito, e a clareza é condição para que o sistema possa evoluir com segurança.

2.2 Modelagem com a linguagem unificada

A Linguagem de Modelagem Unificada é uma especificação mantida pelo Object Management Group (2017) para visualizar, especificar e documentar sistemas de software. Ela organiza seus diagramas em estruturais, como os de classes e de componentes, que descrevem os elementos do sistema e suas relações, e comportamentais, como os de casos de uso, de atividades, de estados e de sequência, que descrevem o que o sistema faz e como seus elementos interagem ao longo do tempo.

2.3 Autenticação por token

O JSON Web Token, definido pela Internet Engineering Task Force (2015), é um formato compacto para representar declarações transferidas entre duas partes. O token reúne informações como a identidade do usuário e o prazo de validade e é assinado digitalmente, de modo que o servidor pode confirmar sua integridade sem consultar um registro de sessões. Essa característica permite que a aplicação não mantenha estado de sessão no servidor.

2.4 Modelos de linguagem e saída estruturada

Modelos de linguagem produzem, por natureza, texto livre, o que dificulta o uso de suas respostas por um programa. A documentação do Spring AI (SPRING, 2026) descreve o recurso de saída estruturada, que anexa à requisição a descrição do formato esperado e converte a resposta do modelo diretamente em um objeto da aplicação. O recurso reduz o trabalho de interpretação, mas não elimina o risco de o modelo preencher campos com valores inventados, o que justifica manter a revisão humana sobre qualquer dado extraído.

2.5 Testes automatizados

Beck (2002) propõe o desenvolvimento guiado por testes, em que o teste é escrito antes do código que o satisfaz, e argumenta que os testes servem tanto para verificar quanto para orientar o projeto. Na prática, os testes automatizados costumam ser organizados em níveis: muitos testes de unidade, rápidos e isolados; uma quantidade menor de testes de integração, que exercitam os componentes em conjunto; e poucos testes de ponta a ponta, que percorrem a aplicação como o usuário a utiliza.

Para os testes de integração, a biblioteca Testcontainers (2026) permite criar instâncias descartáveis de bancos de dados em contêineres, de modo que os testes sejam executados contra o mesmo banco usado em produção. Para os testes de ponta a ponta, o Playwright (MICROSOFT, 2026) automatiza navegadores reais e permite verificar a interface como um usuário a perceberia.

3 METODOLOGIA

3.1 Tipo de pesquisa

Trata-se de uma pesquisa de natureza aplicada, voltada à solução de um problema prático (GIL, 2010), com abordagem qualitativa e delineamento de desenvolvimento de software. O trabalho foi conduzido pelos cinco integrantes da equipe e teve como produto o próprio sistema, acompanhado de sua documentação técnica.

3.2 Etapas e cronograma

O desenvolvimento foi planejado em oito etapas sucessivas, apresentadas no Quadro 1. As quatro primeiras concentram o levantamento de requisitos, a modelagem e a documentação; as seguintes destinam-se à configuração do ambiente, à implementação, aos testes e à apresentação. A ordem obedece à dependência técnica entre as etapas: o registro de presença e a emissão de certificados, por exemplo, dependem da existência de inscrições confirmadas.

Quadro 1 – Cronograma de atividades

Etapa	**Período previsto**	**Entregável**	**Situação**
1. Levantamento de requisitos e definição de escopo	01/09 a 15/09/2026	Requisitos funcionais e não funcionais	Concluída
2. Modelagem UML	16/09 a 05/10/2026	Casos de uso, classes, estados, atividades, interação e componentes	Concluída
3. Modelagem de dados	06/10 a 15/10/2026	Modelo lógico, criação física e dicionário de dados	Concluída
4. Prototipação e documentação	16/10 a 23/10/2026	Protótipos e documentação do software	Concluída
5. Configuração do ambiente	24/10 a 02/11/2026	Repositório, migrações, integração contínua e ambientes publicados	Parcial
6. Implementação do núcleo	03/11 a 23/11/2026	Autenticação, eventos, inscrições e fila de espera	Concluída
7. Presença, certificação e inteligência artificial	24/11 a 07/12/2026	Check-in, certificados e extração automática de dados	Concluída
8. Testes, ajustes e apresentação	08/12 a 15/12/2026	Testes aprovados e sistema apresentado	Em andamento

Fonte: Os autores (2026).

Na execução, a implementação foi antecipada e conduzida em paralelo à modelagem, de modo que o sistema já se encontrava implementado, testado e publicado ao final de outubro de 2026. A etapa 5 aparece como parcial porque o fluxo de integração contínua foi configurado no repositório, mas ainda não é executado automaticamente; da etapa 8, resta a apresentação.

3.3 Tecnologias

As tecnologias foram escolhidas pela maturidade, pela documentação disponível e pela existência de planos gratuitos de hospedagem, e estão resumidas no Quadro 2.

Quadro 2 – Tecnologias empregadas

Camada	**Tecnologias**
Servidor	Java 21, Spring Boot 4.1, Spring Security e Spring AI
Banco de dados	PostgreSQL, com a estrutura criada por migrações versionadas do Flyway
Certificados	Biblioteca iText, com o documento gerado em memória a cada download
Interface	React com TypeScript e Tailwind CSS, empacotados pelo Vite
Inteligência artificial	Modelos de linguagem do Groq, com OpenRouter, OpenAI e Ollama como alternativas configuráveis
Testes	JUnit, Mockito, Testcontainers e Playwright
Publicação	Docker; banco no Neon, servidor no Render e interface na Vercel

Fonte: Os autores (2026).

3.4 Especificação e rastreabilidade

A especificação definiu dez requisitos funcionais, cinco requisitos não funcionais e onze regras de negócio, cada um com identificador próprio. Esse identificador acompanha a regra até o código e os testes: o trecho que a implementa a cita em comentário, e o teste correspondente a carrega no nome. A rastreabilidade assim obtida permitiu conferir, a cada mudança, se a especificação continuava atendida. A modelagem produziu diagramas de casos de uso, classes, estados, atividades, interação e componentes, todos conferidos contra o código ao final do projeto.

O código adotou nomes em português, que aproximam o programa do vocabulário do domínio, e funções de responsabilidade única. O histórico de versões seguiu mensagens padronizadas, que identificam a natureza de cada alteração.

3.5 Verificação e publicação

A verificação seguiu a organização em níveis descrita na seção 2.5. Os testes de unidade validam as regras isoladamente; os de integração sobem um banco PostgreSQL descartável e exercitam as rotas da aplicação; e os de ponta a ponta percorrem a interface em um navegador real, a partir do cadastro do evento até a validação do certificado. A escrita dos testes acompanhou a implementação, e cada regra de negócio crítica tem ao menos um teste com seu identificador no nome.

Concluída a implementação, a aplicação foi publicada em serviços gratuitos de nuvem e utilizada pela equipe em situação real, com organizadores e participantes acessando de computadores e celulares diferentes. Esse uso funcionou como etapa adicional de validação e gerou ajustes discutidos na seção 4.

O código-fonte, a especificação técnica e os diagramas estão disponíveis no repositório do projeto: https://github.com/Jayzmatos22/GerenciadorDeEventos.

4 RESULTADOS E DISCUSSÃO

4.1 Requisitos e regras de negócio

Os requisitos funcionais, listados no Quadro 3, cobrem o ciclo do evento, da criação de conta à avaliação. Os requisitos não funcionais fixaram metas de desempenho, com a lista de eventos carregando em menos de dois segundos mesmo com quinhentos eventos cadastrados; de resiliência, com o cadastro manual sempre disponível; de segurança, com senhas guardadas apenas como resumo criptográfico e chaves de acesso nunca expostas no navegador; de autorização, com cada evento gerenciado apenas por quem o criou; e de usabilidade, com funcionamento em telas de celular a partir de 360 pixels de largura.

Quadro 3 – Requisitos funcionais

Requisito	**Descrição**	**Ator**
RF-01	Criar conta e fazer login	Visitante
RF-02	Criar evento manualmente, com fotos, datas e vagas	Organizador
RF-03	Criar evento a partir de um texto, com preenchimento por inteligência artificial	Organizador
RF-04	Pesquisar e filtrar eventos	Participante
RF-05	Garantir vaga em um evento	Participante
RF-06	Entrar na fila de espera e subir na fila em caso de desistência	Sistema
RF-07	Cancelar a própria inscrição	Participante
RF-08	Registrar presença na hora do evento	Organizador
RF-09	Gerar certificado para quem compareceu, após o término do evento	Sistema
RF-10	Enviar avaliação e gerar resumo das avaliações	Participante e sistema

Fonte: Os autores (2026).

As regras de negócio, resumidas no Quadro 4, concentram as decisões que mais afetam o comportamento do sistema e foram a principal referência para os testes automatizados.

Quadro 4 – Regras de negócio

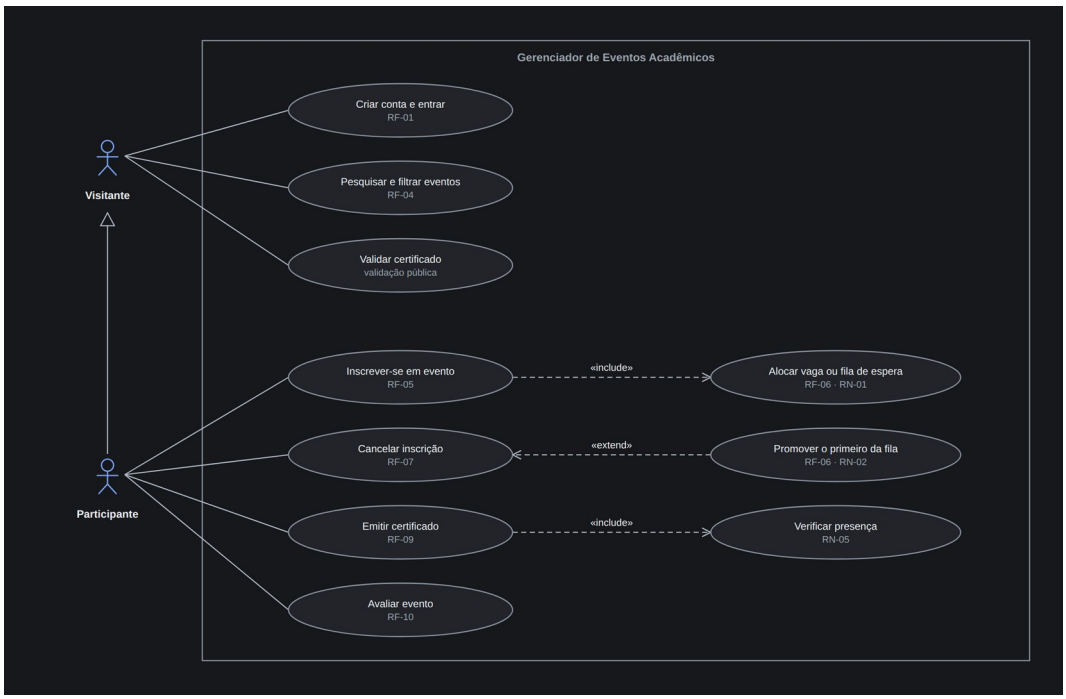
Regra	**Descrição resumida**
RN-01	Inscrição confirmada enquanto houver vaga; com o evento lotado, o participante entra no fim da fila de espera
RN-02	Cancelar uma inscrição confirmada promove automaticamente o primeiro da fila
RN-03	Uma única inscrição ativa por pessoa e evento; reinscrição permitida após cancelar

Regra	Descrição resumida
RN-04	Presença registrada só pelo organizador do evento, entre o início e 24 horas após o fim, para inscrições confirmadas
RN-05	Certificado só com presença registrada e após o término do evento, pelo horário ou pelo encerramento antecipado; emissão idempotente, com código de autenticidade
RN-06	Avaliação só após o fim do evento, por quem teve presença, uma por inscrição
RN-07	Resumo das avaliações por inteligência artificial só a partir de um mínimo de avaliações
RN-08	Somente o criador do evento pode gerenciá-lo
RN-09	Eventos encerrados ou cancelados não aceitam edição nem novas inscrições
RN-10	A falha da inteligência artificial não impede o cadastro manual
RN-11	Senhas com no mínimo 8 caracteres, guardadas apenas como resumo criptográfico

Fonte: Os autores (2026).

A Figura 1 apresenta os casos de uso do visitante e do participante. O participante herda as funcionalidades do visitante, como pesquisar eventos e validar certificados, e acrescenta as que dependem de conta. As relações de inclusão tornam obrigatórias a alocação de vaga na inscrição e a verificação de presença na emissão do certificado, enquanto a relação de extensão indica que a promoção do primeiro da fila só ocorre quando um cancelamento libera uma vaga. O organizador tem diagrama próprio, no qual o preenchimento por inteligência artificial aparece como extensão da criação de evento.

Figura 1 – Diagrama de casos de uso do visitante e do participante

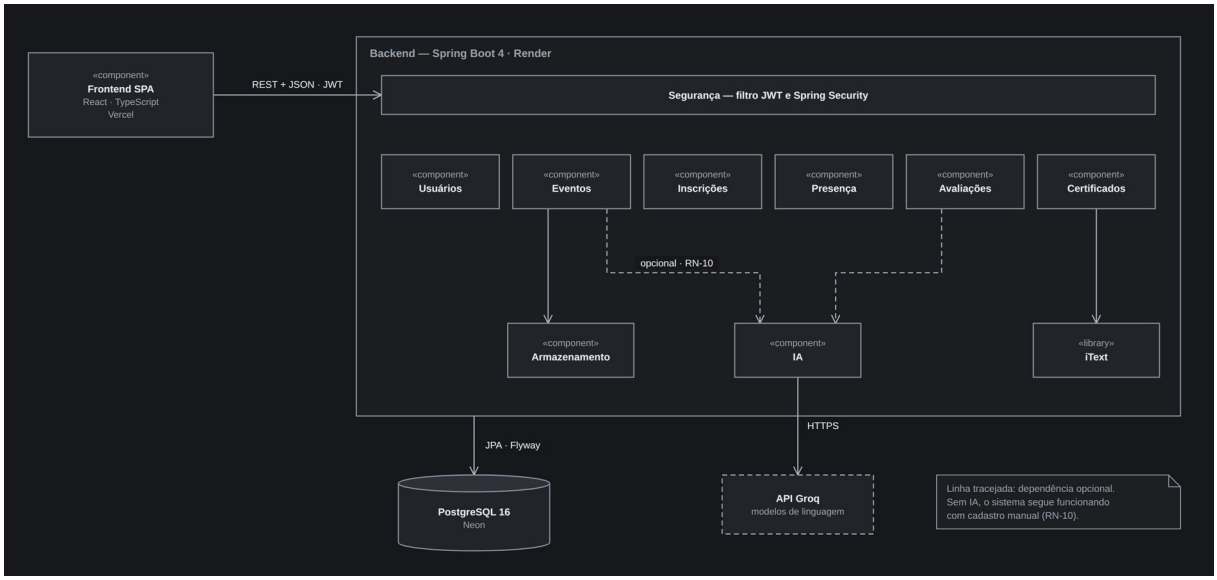


Fonte: Os autores (2026).

4.2 Arquitetura do sistema

A Figura 2 mostra a arquitetura adotada, que segue a divisão em camadas discutida na seção 2.1. A interface, publicada separadamente, comunica-se com o servidor por mensagens no formato JSON, e toda requisição passa por um filtro de segurança que valida o token antes de alcançar os módulos de domínio: usuários, eventos, inscrições, presença, avaliações e certificados. A separação entre interface e servidor permitiu publicar cada parte em uma plataforma especializada, ao custo de declarar explicitamente as origens autorizadas a consumir o servidor.

Figura 2 – Diagrama de componentes



Fonte: Os autores (2026).

As dependências do módulo de inteligência artificial aparecem tracejadas porque são opcionais: sem elas, o sistema continua operando com o cadastro manual. Outra decisão merece destaque: a verificação de que um usuário é dono do recurso que tenta alterar, como o organizador que edita um evento, é feita dentro dos serviços, e não apenas pelo papel do usuário. O papel diz se a pessoa pode, em tese, executar a ação; a regra de propriedade decide se ela pode executá-la sobre aquele evento específico.

4.3 Modelo de dados

O modelo de dados reúne dez tabelas: usuários e seus papéis; eventos e suas fotos; inscrições; presenças; certificados; avaliações; resumos de avaliação; e o

registro das interações com a inteligência artificial. A inscrição deixa de ser uma simples associação entre usuário e evento e concentra a situação da vaga e a posição na fila. Presença, certificado e avaliação ligam-se à inscrição, no máximo um de cada, o que reflete no banco as regras de que cada participação gera uma única presença, um único certificado e uma única avaliação.

Duas restrições merecem destaque. Um índice único parcial impede que a mesma pessoa tenha duas inscrições ativas no mesmo evento, mas permite a reinscrição depois de um cancelamento. E o evento carrega um número de versão, usado no controle de concorrência descrito a seguir. A estrutura é criada por migrações versionadas, aplicadas automaticamente na inicialização, de modo que o banco de desenvolvimento, o de testes e o de produção tenham sempre a mesma forma.

4.4 Inscrições e fila de espera

O controle de vagas exigiu atenção especial à concorrência. Quando duas pessoas disputam a última vaga ao mesmo tempo, as duas requisições calculam, de início, que há vaga disponível. A solução adotada usa o controle otimista de versão do evento: as duas transações tentam gravar a mesma versão, a perdedora falha e é repetida, já enxergando a vaga ocupada, e termina na fila de espera. Um teste de integração dispara duas requisições simultâneas contra o mesmo evento e confirma esse comportamento.

O cancelamento de uma inscrição confirmada promove o primeiro da fila na mesma transação e renumera as posições restantes, de modo que a fila nunca fica com lacunas. O cancelamento de quem ainda está na fila não promove ninguém, apenas reorganiza as posições.

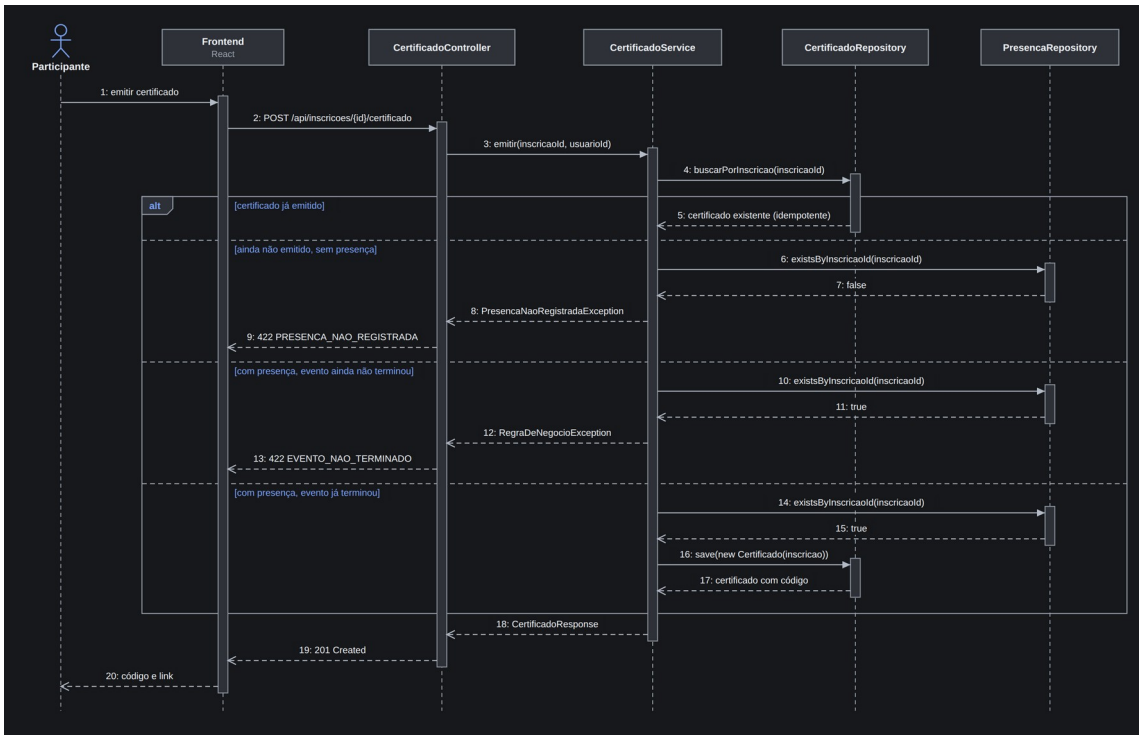
4.5 Presença e certificação

A presença é registrada pelo organizador durante o evento e só é aceita para inscrições confirmadas, dentro da janela que vai do início até 24 horas após o término. O registro é idempotente: uma segunda leitura do mesmo participante devolve a

presença existente em vez de produzir erro, o que é útil quando o registro é feito por leitura de código.

A regra de certificação foi a que mais evoluiu durante o projeto. Na versão inicial, a existência de presença bastava para a emissão. O uso do sistema publicado mostrou que isso permitia emitir o certificado logo após o registro de presença, antes de a pessoa ter permanecido no evento. A regra passou então a exigir também o término do evento, seja pelo horário de fim previsto, seja pelo encerramento antecipado feito pelo organizador; eventos cancelados não geram certificado. A Figura 3 apresenta o fluxo de emissão com essas condições.

Figura 3 – Diagrama de interação da emissão do certificado



Fonte: Os autores (2026).

Como mostra a Figura 3, a emissão é idempotente: um certificado já emitido é devolvido sem nova gravação, com o mesmo código de autenticidade. Sem presença, a resposta é a recusa por presença não registrada; com presença, mas antes do fim do evento, a recusa indica que o evento ainda não terminou. O documento em PDF não é gerado nesse momento, e sim a cada download, em memória e sem gravação em disco. A validação é pública: qualquer pessoa informa o código e recebe o nome do participante, o evento, a carga horária e as datas, sem dados de contato.

4.6 Inteligência artificial

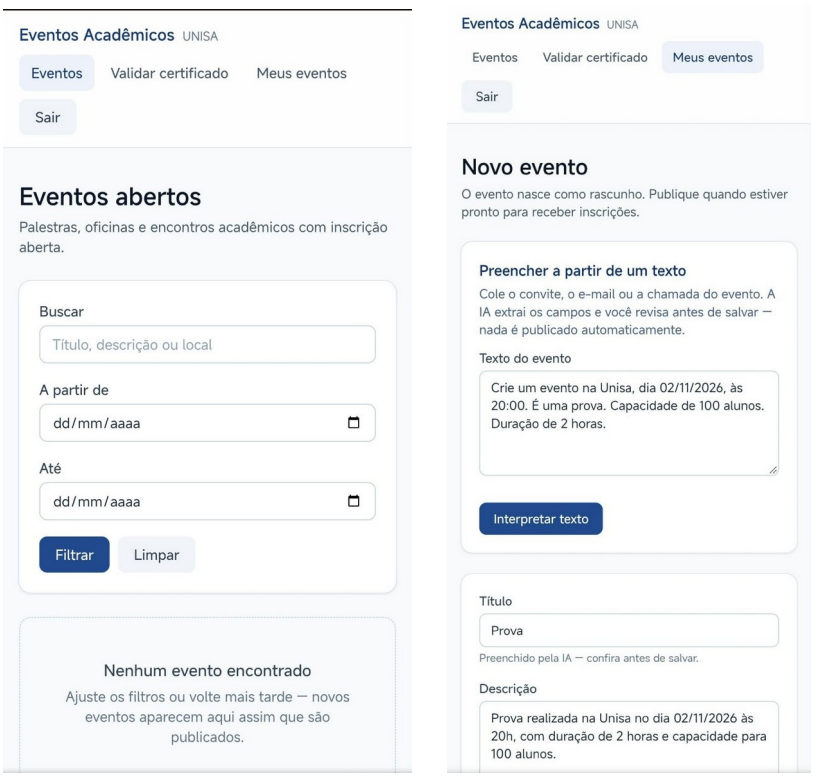
A inteligência artificial é usada em dois pontos. No cadastro, o organizador cola o texto de um convite e o sistema devolve um rascunho com título, descrição, datas, local, carga horária e vagas, por meio do recurso de saída estruturada discutido na seção 2.4. As instruções enviadas ao modelo proíbem deduzir valores ausentes: o que o texto não informa volta vazio, em uma lista própria, e nada é gravado antes da revisão do organizador. No pós-evento, a inteligência artificial sintetiza as avaliações recebidas em um resumo consolidado, gerado apenas a partir de um número mínimo de avaliações.

Como o projeto utiliza serviços gratuitos, a resiliência foi tratada como requisito. A aplicação aceita uma lista de modelos de linguagem e a percorre até que um responda, iniciando de um ponto diferente a cada chamada; um modelo que a conta não pode usar é retirado da rotação. Cada chamada tem limite de 20 segundos e fica registrada, com ou sem sucesso, para auditoria. Esgotadas as tentativas, a interface informa a indisponibilidade e mantém o formulário manual intacto, o que atende à regra que impede a falha da inteligência artificial de bloquear o cadastro.

4.7 Interface

A interface foi projetada em duas etapas: protótipos de baixa fidelidade, que definiram a estrutura das telas, e a interface implementada, apresentada na Figura 4. A listagem de eventos oferece filtros por texto e período, e o cadastro de evento traz, no topo, a área em que o organizador cola o texto do convite. Os campos preenchidos pela inteligência artificial recebem uma indicação explícita de que devem ser conferidos antes de salvar, o que materializa na tela a exigência de revisão humana. O layout em coluna única e os elementos dimensionados para toque atendem ao requisito de funcionamento em celulares.

Figura 4 – Listagem de eventos e cadastro assistido por inteligência artificial



Fonte: Os autores (2026).

4.8 Testes e publicação

Ao final do projeto, a suíte do servidor reunia 136 testes automatizados de unidade e de integração, e a suíte de ponta a ponta somava onze cenários executados em navegador real, cobrindo o catálogo, a fila de espera, o fluxo completo do evento ao certificado, a indisponibilidade da inteligência artificial, a liberação do certificado após o encerramento e o uso de contas diferentes em abas do mesmo navegador.

Os testes revelaram falhas que dificilmente seriam percebidas em verificações manuais. A primeira era de fuso horário: o servidor usava o relógio do sistema operacional, e, em um servidor no horário universal, as janelas de registro de presença e de avaliação ficariam deslocadas em três horas. A segunda era de ordem das respostas: uma busca antiga que chegasse depois da nova repunha a lista anterior e desfazia o filtro aplicado.

O uso do sistema publicado acrescentou três ajustes. Além da regra do certificado, já descrita, constatou-se que a sessão guardada de forma compartilhada entre as abas do navegador fazia uma conta substituir a outra quando organizador e participante eram usados lado a lado; a sessão passou a ser guardada por aba.

Verificou-se, ainda, que eventos em rascunho ou marcados como em andamento não aparecem no catálogo público, o que confundia quem os procurava; a tela do organizador passou a informar se o evento está visível.

4.9 Discussão

Em relação à primeira parte da hipótese, os resultados indicam que ela se confirma: o fluxo que vai da criação do evento à validação pública do certificado é executado de ponta a ponta em uma única aplicação, e as regras que garantem a confiabilidade dos registros, como o controle de vagas, a exigência de presença e a validação pública, são aplicadas pelo próprio sistema e estão cobertas por testes. A inteligência artificial mostrou-se útil como apoio, desde que tratada como sugestão sujeita a revisão e cercada de um caminho alternativo.

A segunda parte da hipótese, a redução do tempo gasto pelos organizadores, não foi medida com usuários reais. Ela permanece, portanto, como expectativa fundamentada na eliminação de etapas manuais, e não como resultado comprovado. Também se observou que o uso real revelou problemas que a especificação não previa, o que reforça o valor de publicar o sistema cedo e observar seu uso.

5 CONCLUSÃO

O trabalho atingiu o objetivo geral de desenvolver uma aplicação web para a gestão completa de eventos acadêmicos. Os objetivos específicos foram cumpridos: o sistema oferece acesso diferenciado para participantes e organizadores, permite criar eventos por formulário ou a partir de texto livre, controla vagas com fila de espera e promoção automática, registra a presença, emite certificados verificáveis publicamente após o término do evento, coleta e resume avaliações e continua operando quando a inteligência artificial está indisponível.

Aprendeu-se, sobretudo, que regras de negócio especificadas com precisão e verificadas continuamente sustentam a evolução do sistema: quando a regra do certificado precisou mudar, os testes que a descreviam apontaram o que precisava ser revisto. Aprendeu-se também que a inteligência artificial agrega valor quando é tratada como apoio supervisionado, com o organizador mantendo a decisão final.

Entre as limitações, destacam-se a ausência de medição do ganho de tempo com usuários reais, o registro de presença em um único momento, que não comprova a permanência do participante até o fim, o armazenamento apenas temporário das fotos de divulgação no plano gratuito de hospedagem e a integração contínua ainda não executada automaticamente. Ficaram fora do escopo a cobrança de inscrições, a divisão do evento em sessões, a submissão de trabalhos acadêmicos e as notificações por correio eletrônico.

Como continuidade, sugere-se avaliar o sistema com organizadores e participantes em eventos reais, medindo o tempo de cadastro e de certificação; registrar também a saída dos participantes, para comprovar a permanência; armazenar as imagens em um serviço próprio para objetos; e incluir o envio de notificações sobre inscrições, promoções da fila e disponibilidade do certificado.

O código-fonte, a especificação técnica e os diagramas do projeto permanecem disponíveis para consulta e continuidade no repositório https://github.com/Jayzmatos22/GerenciadorDeEventos.

REFERÊNCIAS

BECK, Kent. Test-driven development: by example. Boston: Addison-Wesley, 2002.

FOWLER, Martin. Patterns of enterprise application architecture. Boston: Addison-Wesley, 2002.

GIL, Antonio Carlos. Como elaborar projetos de pesquisa. 2. ed. São Paulo: Atlas, 2010.

INTERNET ENGINEERING TASK FORCE. RFC 7519: JSON Web Token (JWT). [S. l.]: IETF, 2015. Disponível em: https://datatracker.ietf.org/doc/html/rfc7519. Acesso em: 25 out. 2026.

MARTIN, Robert C. Clean code: a handbook of agile software craftsmanship. Upper Saddle River: Prentice Hall, 2008.

MICROSOFT. Playwright documentation. [S. l.], 2026. Disponível em: https://playwright.dev/docs/intro. Acesso em: 25 out. 2026.

OBJECT MANAGEMENT GROUP. OMG Unified Modeling Language (OMG UML): version 2.5.1. Needham: OMG, 2017. Disponível em: https://www.omg.org/spec/UML/. Acesso em: 25 out. 2026.

SPRING. Spring AI reference documentation. [S. l.], 2026. Disponível em: https://docs.spring.io/spring-ai/reference/. Acesso em: 25 out. 2026.

TESTCONTAINERS. Testcontainers for Java. [S. l.], 2026. Disponível em: https://java.testcontainers.org/. Acesso em: 25 out. 2026.